

מספר קורס: 0368.2158

5.5.2024, כ"ז ניסן

סמסטר א', מועד ב', תשפ"ד

פתרון מבחן במבני נתונים

ד"ר רני הוד, פרופ' יוסי עזר, נתן גייר, דני דורפמן, שחר לבקוביץ

משך המבחן שלוש שעות

הבחינה ללא מחשבים/מחשבוניס. מותר להביא דף דו נוסחאות דו צדדי.

הוראות כלליות:

- יש לרשום מספר ת.ז. ומספר מחברת בראש כל דף.
- מותר להביא דף נוסחאות דו צדדי. ניתן להביא דף דו צדדי נוסף לקבוצה 28.
- המחברות הן לטייטה בלבד ולא תיבדקנה.
- מותר להשתמש באלגוריתמים שנלמדו בכיתה כקופסאות שחורות.
- אין לכתוב אלגוריתמים בפסאודו-קוד אלא להסביר (באופן משכנע) את הרעיון הכללי.
- כתבו תשובות קצרות ומדויקות. תשובות לא ממוקדות, ארוכות, מסובכות או מסורבלות יקבלו ניקוד חלקי גם אם הן נכונות. עם זאת, תשובה ללא נימוק מדויק לא תזכה בנקודות.
- סיבוכיות זמני ריצה יש לכתוב כחסם הדוק ככל שתוכלו, בכתב $O()$. יש לנתח ולנמק את הסיבוכיות.
- אלא אם נאמר אחרת, על האלגוריתמים להיות דטרמיניסטיים (אלא אם כן כתוב בתוחלת) וזמני הריצה הם worst-case (אלא אם כן כתוב אמורטייזד).
- יש לענות על כל שאלה במסגרת המקום המוקצה, בלי חריגות פרט לשימוש ב"דף החירום" הנוסף.
- כשמבקשים חציון של מספר איברים זוגי, הכוונה לחציון תחתון.
- בכל השאלות ניתן להניח שהמפתחות ייחודיים, אלא אם נאמר אחרת.

בהצלחה!!!

שאלה 1 (20 נקודות)

א.

[Redacted text for part A]

ב.

[Redacted text for part B]

הטריק הקלאסי בשאלות מסוג זה הוא לבודד משתנים:

$$j - i = |a_j - a_i| \text{ אם } j - i = a_i - a_j \text{ או } j - i = a_j - a_i$$

$$a_i - i = a_j - j \quad \vee \quad a_i + i = a_j + j$$

כמו כן, נשים לב כי שני התנאים לא יכולים להתקיים בו-זמנית (מה שימנע ספירה כפולה).

סעיף א

אלגוריתם – נעזר ב-2 מילונים במימוש *hash table* בגודל n בשיטת *chaining*. נסמן את המילונים ב- D_1, D_2 . הערך שמתאים למפתח ימנה את כמות הפעמים שהכנסנו את המפתח.

עבור $i = 1 \dots n$:

- נכניס את $a_i - i$ ל- D_1 עם ערך 1 אם הוא לא שם כבר, אחרת נגדיל את הערך שלו.
 - נכניס את $a_i + i$ ל- D_2 עם ערך 1 אם הוא לא שם כבר, אחרת נגדיל את הערך שלו.
- לסיום, נעבור על כל הערכים במילונים ולכל ערך x נוסיף לתשובה $\binom{x}{2}$.

סיבוכיות $O(n)$ בתוחלת: כמות לינארית של פעולות על טבלת *hash* בגודל n שעולות זמן קבוע בתוחלת, ולסיום עוד מעבר על הטבלה שעולה $O(n)$ במקרה הגרוע.

סעיף ב

ניצור 2 מערכים, אחד עם הערכים $a_i + i$, השני עם $a_i - i$. נבחין כי כל הערכים שלמים בתחום $\{-n^2 - n, \dots, n^2 + n\}$ ולכן אנו יודעים למיין אותם בזמן לינארי עם *radix sort*. לסיום נעבור על כל אחד מהמערכים הממוינים, במידה ויש רצף של x מפתחות זהים, נוסיף לתשובה $\binom{x}{2}$.

סה"כ סיבוכיות $O(n)$.

שאלה 2 (20 נקודות)

א.

.

ב.

א. יש מספר פתרונות אפשריים: **פתרון ראשון**: עץ AVL שיכיל את המפתחות וערמת מקסימום שתכיל את השכיחויות של המפתחות ומצבעים דו כוונים ביניהם.

$Insert(k)$: נחפש את k . אם k לא בעץ אז נכניס אותו לעץ ולערמה עם $value = 1$,

אחרת נגדיל את ערכו בערמה ונעשה $heapify\ up$.

$delete(k)$: נחפש את k . אם $value = 1$ אז נמחק את הצומת המתאים ל k מהעץ

ומהערימה. אחרת נעשה נקטין את $value$ ב 1 בערמה ונבצע $heapify\ down$.

$freq(k)$: נחפש את k ונחזיר את ה $value$ שלו.

$mostFreq()$: נחזיר הערך בראש הערמה ואת המפתח המתאים לו

פתרון שני: נעזר בעץ AVL אליו נכניס את המפתחות. לכל מפתח נצמיד $value$ שמציין כמה

פעמי המפתח הוכנס. בנוסף נתחזק שדה בכל צומת של $maxVal$: מהו ה $value$ המקסימלי

בתת העץ של הצומת. נבחין כי ניתן לחשב את השדה על סמך ערך השדה אצל הבנים:

$node.maxVal = \max\{node.left.maxVal, node.right.maxVal, node.val\}$

לכן, לפי משפט מהתרגול, ניתן לתחזק את השדה $maxVal$. בהוספה אם k לא בעץ אז נכניס

אותו עם $value = 1$, אחרת $value++$. במחיקה נחפש את k . אם $value = 1$ אז

נמחק את k מהעץ. אחרת נעשה נקטין את $value$ ב 1. $freq(k)$: נחפש את k ונחזיר את

ה $value$ שלו. $MostFreq()$: נחזיר הערך בראש העץ ואת המפתח המתאים לו (נשמור).

ב. נשתמש במילון מבוסס $hashing$ במימוש $chaining$ עם טבלה בגודל n . כמו בסעיף א, לכל

מפתח נתאים גם $value$ שיהיה מונה לכמות הפעמים שהוכנס המפתח. בנוסף נתחזק שדה

גלובאלי $maxVal$ של השכיחות המרבית.

בגלל שאין מחיקות אז $maxVal$ רק יכול לגדול.

$insert(k)$: אם k לא במילון אז נכניס אותו עם $value = 1$, אחרת $value++$.

$delete(k)$: נחפש את k . אם $value = 1$ אז נמחק את k מהמילון. אחרת נעשה נקטין

את $value$ ב 1.

$freq(k)$: נחפש את k ונחזיר את ה $value$ שלו.

$mostFreq()$: נחזיר $maxVal$.

שאלה 3 (20 נקודות)

א.

ב.

א. משתמשים ברמז. נניח בשלילה שלכל i מתקיים $B[i+1] - B[i] > \frac{M_A - m_A}{n-1}$ אם

נסכום את הביטוי עבור $1 \leq i \leq n-1$ ונקבל

$$M_A - m_A = B[n] - B[1] > (n-1) \frac{M_A - m_A}{n-1} = M_A - m_A$$

בסתירה.

ב. אלגוריתם רקורסיבי:

בהתחלה טווח האינדקסים הוא $[1, n]$.

נניח שכעת טווח האינדקסים הוא $[l, r]$.

מקרה בסיס: אם $j = i + 1$, התשובה תהיה האיברים $select(A, l), select(A, r)$.

מקרה כללי ($j - i \geq 2$):

1. נחשב את האמצע $m = l + \left\lfloor \frac{r-l}{2} \right\rfloor$

2. נחשב $x_m = select(m), x_r = select(r), x_l = select(l)$

3. נחשב $a = \frac{x_m - x_l}{m-l}, b = \frac{x_r - x_m}{r-m}$.

4. אם $a \leq b$ אז נמשיך ברקורסיה על $[l, m]$ ואחרת על $[m, r]$.

הנכונות נובעת מהטענה הבאה + סעיף א:

טענה: לכל $1 \leq i < j < k \leq n$ מתקיים (כאשר המערך B הוא ממוין)

$$\min \left\{ \frac{B[k] - B[j]}{k-j}, \frac{B[j] - B[i]}{j-i} \right\} \leq \frac{B[k] - B[i]}{k-i}$$

שאלה 4 (20 נקודות)

א. מבנה הנתונים: עץ AVL רגיל.

הכנסה, מחיקה וחיפוש ממומשים כרגיל ב $O(\log n)$.

: $AlternatingDeletions(k)$

1. נמיר את העץ למערך ממויין על ידי $in-order$.
2. נעביר בסדר ממויין את המפתחות הרלוונטיים למערך חדש.
3. נמיר את המערך הממויין החדש לעץ AVL עם האלגוריתם מהתרגול.

סיבוכיות שלושת השלבים הינה לינארית.

ב. מבנה הנתונים: עץ AVL עם תחזוקת שדה $size$ (בשביל $selection$ יעיל)

הכנסה, מחיקה וחיפוש ממומשים כרגיל ב $O(\log n)$.

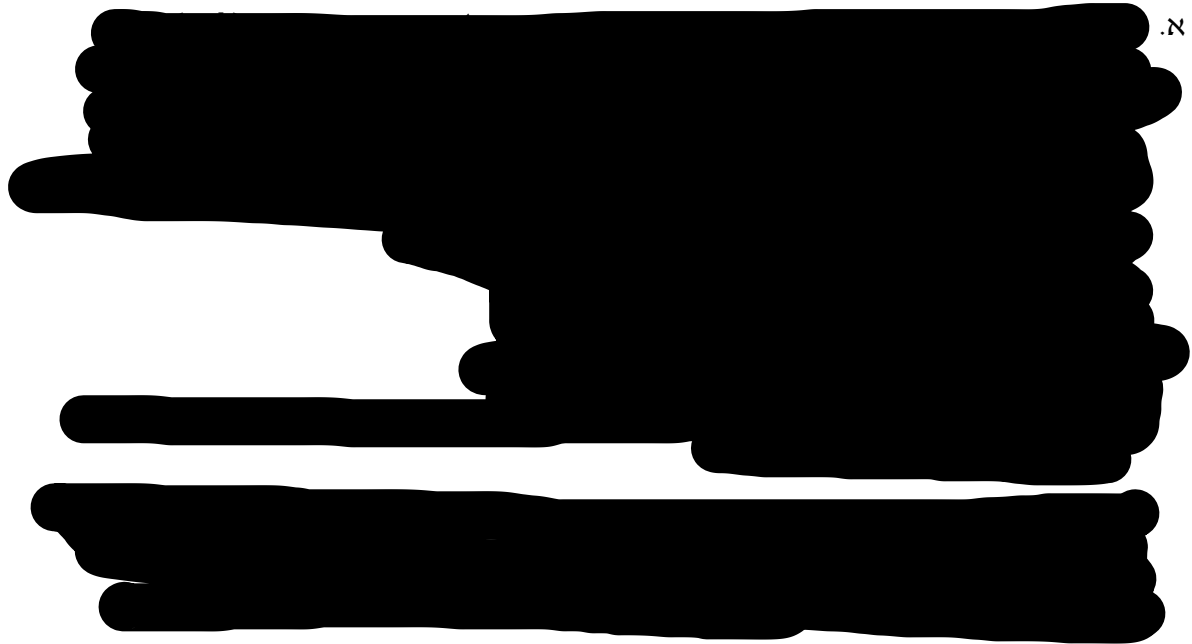
: $AlternatingDeletions(k)$

1. נבצע $select(i \cdot \sqrt{n})$ עבור $i = 1 \dots \sqrt{n}$ ונשמור מצביעים לצמתים המתאימים.
2. נבצע $split$ על כל האיברים הללו. נקבל רשימה של עצים.
3. נבצע $join$ לכל העצים האי זוגיים.

סה"כ בצענו $\Theta(n)$ פעולות $select, split, join$.

הבהרות: הפעולה $split$ מפצלת ל 2 עצים וצומת והפעולה $join$ מקבלת 2 עצים בצומת. לכן נעזר בצמתים שקיבלנו מהפיצולים כאשר נבצע את סדרת פעולות ה $join$.

שאלה 5 (20 נקודות)

א. 

ב. 

א. מבנה הנתונים: 2 ערימות פיבונאצ'י, אחת מינימום והשנייה מקסימום: H_m, H_M שישמרו את קצות הקטעים. בנוסף נתחזק ערימת פיבונאצ'י מקסימום לאורכי הקטעים H_L . שלושת הערימות יתחזקו בנוסף לkeys גם value שהוא מצביע לקטע עצמו. בנוסף 3 הערימות יתחזקו מצביעים דו כוונים.

$Insert(l, r)$: נבצע $H_m.insert(l), H_M.insert(r), H_L.insert(r - l)$ ונתחזק מצביעים דו כוונים.

$Delete(l)$: נמחק באמצעות המצביע ב 2 הערימות.

$Left()$: נחזיר $H_m.min$.

$Right()$: נחזיר $H_M.max$.

$Largest()$: נחזיר $H_L.max$.

$Expand(l, c)$: באמצעות המצביעים הדו כוונים, נגיע בשלוש הערימות לצמתים הרלוונטיים ונשנה אותם בהתאם (על ידי ביצוע increase key/ decrease key באמורטיזד קבוע).

ב. לא קיים: נניח בשלילה שקיים מבנה נתונים שכזה. נציג רדוקציה למיון. בהינתן מערך $a_1, a_2, \dots, a_n$ שנרצה למיין, נבצע את התהליך הבא: ראשית אם התחום הסדור ממנו נלקחו איברי המערך הוא M אז נרחיב את התחום הסדור ונוסיף לו איבר חדש מקסימלי ∞ . אלגוריתם המיון יהיה:

1. עבור $i = 1 \dots n$: נכניס את הקטע (a_i, ∞) למבנה הנתונים.
 2. עבור $i = 1 \dots n$: נדפיס את $left()$ לפלט של המערך הממויין ואז נמחוק אותו ממבנה הנתונים על ידי $delete()$.
- סה"כ ביצענו $\Theta(n)$ פעולות $insert, left, delete$ שכולן בזמן אמורטייזד קבוע ולכן הצגנו אלגוריתם מיון המבצע $O(n)$ השוואות. בסתירה.

טעויות נפוצות.**שאלה 4**

סעיף ב – היו פתרונות שניסו לתחזק $\sqrt{n}$ עצים בגדלים שווים ולהעביר איברים בין עצים סמוכים. הפתרון לא פועל משתי סיבות:

א. העצים לא מחולקים בדיוק לפי האינדקסים המבוקשים.

ב. המנגנון של להעביר מפתחות בין עצים (כדי שהגדלים יהיו שווים) הוא יקר מדי. למשל אם כל העצים פרט לאחרון הם בגודל $\sqrt{n}$ והעץ האחרון בגודל $\sqrt{n} - 1$. אם נכניס עכשיו מפתח לעץ הראשון אז נצטרך לעשות סדרה של העברת מפתחות עד לעץ האחרון, מה שהופך את עלות ההכנסה ל $O(\sqrt{n} \log n)$ במקום $O(\log n)$.

בהצלחה!!!